OpenMiniCrew

Tools Guide

How to create and extend tools for the framework

Auto-discover

Registry scans tools/

2 Ways to Use

/command + free-text

No Core Edits

Just 1 file in tools/

How Tool System Works

1. **Create .py file in tools/**
   Filename != base.py, registry.py, __init__.py

2. **Registry auto-discovers on startup**
   importlib + inspect finds BaseTool subclasses

3. **Ready to use in 2 ways**
   /command (no LLM token) + free-text (LLM function calling)

No need to edit any core files

5 Steps to Add a New Tool

1	**Create file**	`touch tools/my_tool.py`	Must not be base.py, registry.py, __init__.py
2	**Write class**	`class MyTool(BaseTool):`	Set name, description, commands
3	**Dependencies**	`pip install some-lib`	(Optional) Add to requirements.txt
4	**Config**	`MY_KEY=xxx  # .env`	(Optional) Add to .env + core/config.py
5	**Restart bot**	`python main.py`	Log shows: Registered tool: my_tool

Basic Template

tools/my_tool.py

```python
from tools.base import BaseTool
from core import db

class MyTool(BaseTool):
    name = "my_tool"
    description = "LLM uses this to decide"
    commands = ["/mytool"]
    direct_output = True

    async def execute(self, user_id, args="", **kwargs):
        try:
            result = do_something(args)
            db.log_tool_usage(user_id, self.name, status="success")
            return result
        except Exception as e:
            db.log_tool_usage(user_id, self.name, status="failed")
            return f"Error: {e}"

    def get_tool_spec(self):
        return {"name": self.name, ...}
```

Example 1: Weather Tool

Easy / Free

Open-Meteo API (free, no API key)

How it works:

- Parse city name from args
- Lookup coordinates from CITIES dict
- Call Open-Meteo API (free, no key)
- Format temperature, humidity, wind
- Return formatted string to user

Usage:

```
/weather bangkok
/weather chiang mai
"How is the weather today?"
```

Pattern:

```
Parse args
  -> Call free API
  -> Format output
  -> Return string
```

No API key, no billing, no dependencies beyond requests

Example 2: Traffic Tool

Medium / API Key

Setup:

1. Add GOOGLE_MAPS_API_KEY to .env
2. Add config to core/config.py
3. Create tools/traffic.py

Usage:

```
/traffic Siam to Silom
/traffic Home to Airport
"How long Siam to Silom?"
```

Google Maps vs Free Alternatives

API	Free Tier	Billing	Real-time Traffic
Google Maps	$200/mo (~40K req)	Yes (credit card)	Yes
OpenRouteService	2,500 req/day	No	Patterns only
Mapbox	100K req/mo	No	Patterns only
Foursquare (places)	100K req/day	No	N/A

Recommended free combo: Foursquare (places) + OpenRouteService (routes)

Example 3: Places Tool (Google Places API)

Medium / API Key

Text Query from args → Places API Text Search → Format results → Formatted summary

```
Key pattern: location bias + open_now filter

resp = requests.post(
    "https://places.googleapis.com/v1/places:searchText",
    headers={"X-Goog-Api-Key": GOOGLE_MAPS_API_KEY, ...},
    json={"textQuery": args, "openNow": open_now, ...},
    timeout=10,
)
return format_places(data["places"])
```

Key Features

Feature	Detail	Commands
Location Bias	GPS / Bangkok fallback	/places, /nearby, /search
Open Now Filter	Detect "เปิดอยู่" keyword	Requires GOOGLE_MAPS_API_KEY + billing

Example 4: News Summary (LLM-powered)

Easy / Free

RSS Feed
Google News

Raw articles
(10 latest)

LLM summarize
tier="cheap"

Formatted
summary

Key pattern: calling LLM inside a tool

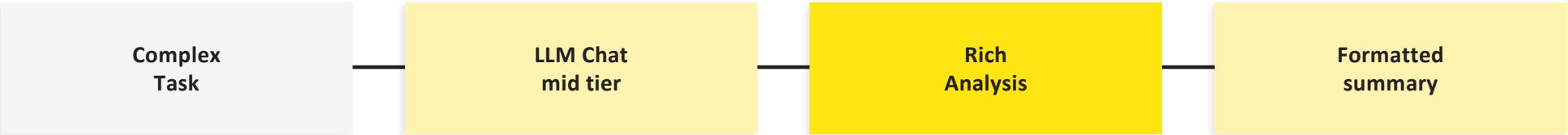
```
resp = await llm_router.chat(
    messages=[{"role":"user", "content": headlines}],
    provider=provider,
    tier=self.preferred_tier,  # "cheap" for news
    system="You are a news summarizer...",
)
return resp["content"]
```

LLM Tier Selection

Tier	Models	Use When
cheap	Haiku / Flash	News summary, formatting, simple tasks
mid	Sonnet / Pro	Email summary, deep analysis, research

Example 5: High-Intelligence Tool (LLM Mid Tier)

Advanced / Mid Tier




```
Key pattern: preferred_tier = "mid" for complex tasks

class ResearchTool(BaseTool):
    preferred_tier = "mid"  # Sonnet / Pro
    async def execute(self, user_id, args, **kwargs):
        resp = await llm_router.chat(
            tier=self.preferred_tier,
)
        return resp["content"]
```

When to Use Mid Tier

Tool	Tier	Reason
`email_summary`	mid	Deep understanding of email content
`work_email`	mid	IMAP + attachment analysis

Key Concepts

execute() Signature

```
async def execute(
  self, user_id, args="", **kwargs
)

user_id = Telegram chat ID
args = text after /command
**kwargs = LLM parameters
Must return string always
```

get_tool_spec()

```
Use self.name (never hardcode)
Clear description = better LLM choice
Use enum for fixed parameter values
parameters.required = []
  if all params are optional
```

direct_output

```
True (default):
  Send result directly to user
  Use for pre-formatted output

False:
  Let dispatcher's LLM summarize raw data
  Rarely used (all current tools = True)
```

Error Handling

```
Always wrap in try/except
Log both success and failure
Return user-friendly error msg
Never let exceptions escape
  out of execute()
```

Deploy Checklist

Required (missing = bug)

- ✓ File in tools/ directory
- ✓ Class inherits BaseTool
- ✓ name, description, commands set
- ✓ execute() returns string always
- ✓ execute() wrapped in try/except
- ✓ get_tool_spec() uses self.name
- ✓ db.log_tool_usage() both paths
- ✓ direct_output set correctly

Recommended

- ✓ Clear description in tool_spec
- ✓ enum for fixed-choice params
- ✓ API key in .env + .env.example
- ✓ New libs in requirements.txt
- ✓ Test via /command first
- ✓ Test via free-text (LLM picks)
- ✓ Update README.md commands
- ✓ Output < 4096 chars (TG limit)

Tool Catalog & Ideas

Tool	Command	API / Source	Status
Email Summary (Gmail)	/email	Gmail API + LLM	**Done**
Work Email (IMAP)	/wm	IMAP + pdfplumber + LLM	**Done**
Traffic	/traffic	Google Maps Directions	**Done**
Places Search	/places	Google Places API	**Done**
News Summary	/news	Google News RSS + LLM	**Done**
Lottery Check	/lotto	lotto.api.rayriffy.com	**Done**
Weather	/weather	Open-Meteo (free)	Idea
Translate	/translate	LLM (no extra API)	Idea
Notes / Reminders	/note	SQLite (built-in)	Idea
Exchange Rate	/fx, /rate, /exchange	Bank of Thailand API	Done
Web Search	/search	Google/SerpAPI	Idea
YouTube Summary	/yt	YouTube Transcript + LLM	Idea
Package Tracking	/track	Thailand Post / Kerry	Idea
Google Calendar	/cal	Calendar API	Idea

Tips for Building Tools

description is everything

LLM uses it to decide which tool to call. If description is vague, LLM picks wrong. Be specific about what the tool does and when to use it.

Use LLM as formatter

Tools fetch raw data, then send to LLM to summarize into natural language. Results are much better than manual formatting. (See email_summary pattern)

Multiple commands per tool

commands = ["/weather", "/w"] lets users type shorter aliases. All map to the same tool.

Test /command first

Direct command bypasses LLM, so you see raw output. Easier to debug than going through function calling.

Keep API keys in .env

Never hardcode in tool files. Always use core/config.py to load from environment.

Thank You!

Now go build something awesome

buymeacoffee.com/ogonan

github.com/kaebmoo/openminicrew

Quick Recap:

1 file = 1 tool

Auto-discovered

/command + free-text

LLM function calling

No core edits needed